



20 JANVIER 2026

POC TERRAFORM & GITHUB

ASRBD - GROUPE 2



1. CONTEXTE ET OBJECTIFS	2
2. DEFINITION DES TECHNOLOGIES.....	2
3. ARCHITECTURE DU POC.....	2
3.1 Composants	2
3.2 Organisation du dépôt GitHub	3
4. Installation de Terraform sur le Bastion.....	3
5. PREPARATION DU DEPOT GITHUB	4
5.1 Création du dépôt.....	4
5.2 Création de la structure	4
6. MODULE VM_PROXMOX	5
7. Environnement d'un groupe (exemple B1-01)	7
7.1 Fichier envs/B1-01/main.tf	7
7.2 Fichier envs/B1-01/variables.tf.....	8
7.3 Fichier envs/B1-01/terraform.tfvars.example	9
8. Utilisation par un groupe.....	9
9. Gestion des états et bonnes pratiques.....	10
10. Annexes – Liens de référence	10

1. CONTEXTE ET OBJECTIFS

Dans le projet de laboratoire, les machines virtuelles utilisées par les B1/B2 doivent être créées de manière automatisée, reproductible et traçable. L'objectif est que chaque groupe puisse déployer ses VMs à partir d'un dépôt GitHub, en exécutant quelques commandes sur le Bastion, plutôt qu'en créant les VMs à la main dans l'interface Proxmox.

Ce POC vise à :

- Mettre en place un dépôt GitHub contenant le code Terraform du laboratoire.
- Permettre à chaque groupe (B1-01, B1-02, B2-01, B2-02) de disposer de son propre environnement Terraform.
- Déployer automatiquement au moins une VM de test sur Proxmox via Terraform, à partir d'un template.

2. DEFINITION DES TECHNOLOGIES

- Terraform : outil d'Infrastructure as Code (IaC) qui décrit l'infrastructure (VMs, réseaux, etc.) dans des fichiers HCL, puis applique ces définitions pour créer ou détruire les ressources.
- Proxmox VE : hyperviseur de virtualisation, accessible via une API REST sécurisée utilisée par le provider Terraform.
- GitHub : plateforme de gestion de code source permettant de versionner les fichiers Terraform, de travailler en équipe et de préparer une future intégration CI/CD (GitHub Actions).

3. ARCHITECTURE DU POC

3.1 Composants

- Bastion (Linux) : exécute Terraform, possède un accès HTTPS à l'API Proxmox.

- Cluster Proxmox : au moins pve1.
- GitHub : dépôt terraform-lab.
- VM Template dans Proxmox : image de base (Ubuntu/Debian) convertie en template (VMID TEMPLATE_VMID).

3.2 Organisation du dépôt GitHub

```

text
terraform-lab/
├── README.md
├── modules/
│   ├── vm_proxmox/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
└── envs/
    ├── B1-01/
    │   ├── main.tf
    │   ├── variables.tf
    │   └── terraform.tfvars.example
    ├── B1-02/
    ├── B2-01/
    └── B2-02/

```

- modules/vm_proxmox contient la définition générique d'une VM Proxmox.
- Chaque dossier envs/B*-** représente un environnement de groupe qui appelle ce module.

4. Installation de Terraform sur le Bastion

Cette étape est effectuée une seule fois par les administrateurs.

Sur un Bastion Ubuntu/Debian :

```
bash
sudo apt update -y
sudo apt install -y software-properties-common gnupg curl

# Ajout du dépôt officiel HashiCorp
curl -fsSL https://apt.releases.hashicorp.com/gpg | sudo apt-key add -
sudo apt-add-repository \
"deb [arch=$(dpkg --print-architecture)] https://apt.releases.hashicorp.com
$(lsb_release -cs) main"

sudo apt update -y
sudo apt install -y terraform

terraform -version
```

La commande terraform -version doit afficher la version installée (ex. Terraform v1.7.x).

5. PREPARATION DU DEPOT GITHUB

5.1 Création du dépôt

- Créer un dépôt GitHub terraform-lab (privé ou public selon la politique).
- Cloner ce dépôt sur le Bastion :

```
bash
git clone https://github.com/ORG/terraform-lab.git
cd terraform-lab
```

5.2 Création de la structure

Créer l'arborescence :

```
bash
mkdir -p modules/vm_proxmox
mkdir -p envs/B1-01 envs/B1-02 envs/B2-01 envs/B2-02
Committer la structure :
bash
git add .
git commit -m "Initialisation dépôt Terraform labo"
git push origin main
```

6. MODULE VM_PROXMOX

Dans modules/vm_proxmox/main.tf :

```
text
terraform {
  required_providers {
    proxmox = {
      source = "Telmate/proxmox"
      version = "~> 3.0"
    }
  }
}

provider "proxmox" {
  pm_api_url      = var.pm_api_url
  pm_api_token_id = var.pm_api_token_id
  pm_api_token_secret = var.pm_api_token_secret
  pm_tls_insecure = true # labo uniquement
}

resource "proxmox_vm_qemu" "vm" {
```

```

name      = var.vm_name
target_node = var.target_node
vmid      = var.vmid
clone     = var.template_vmid
full_clone = true

cores     = var.cores
sockets   = 1
memory    = var.memory_mb

disk {
    size     = var.disk_size
    storage  = var.storage
    type     = "virtio"
}

network {
    model    = "virtio"
    bridge   = var.bridge
}

os_type    = "l26"
onboot     = true
agent      = 1
}

```

Dans modules/vm_proxmox/variables.tf :

```

text
variable "pm_api_url"      { type = string }
variable "pm_api_token_id" { type = string }

```

```
variable "pm_api_token_secret" { type = string }
```

```
variable "vm_name"    { type = string }
```

```
variable "target_node" { type = string }
```

```
variable "template_vmid" { type = number }
```

```
variable "vmid"      { type = number }
```

```
variable "cores"     { type = number }
```

```
variable "memory_mb" { type = number }
```

```
variable "disk_size" { type = string }
```

```
variable "storage"   { type = string }
```

```
variable "bridge"    { type = string }
```

Dans modules/vm_proxmox/outputs.tf :

```
text
```

```
output "vm_name" {
```

```
  value = proxmox_vm_qemu.vm.name
```

```
}
```

```
output "vm_id" {
```

```
  value = proxmox_vm_qemu.vm.vmid
```

```
}
```

7. Environnement d'un groupe (exemple B1-01)

7.1 Fichier envs/B1-01/main.tf

```
text
```

```
module "vm_labo" {
```

```
  source = "../modules/vm_proxmox"
```

```

pm_api_url      = var.pm_api_url
pm_api_token_id = var.pm_api_token_id
pm_api_token_secret = var.pm_api_token_secret

vm_name      = var.vm_name
target_node  = var.target_node
template_vmid= var.template_vmid
vmid         = var.vmid
cores        = var.cores
memory_mb    = var.memory_mb
disk_size    = var.disk_size
storage      = var.storage
bridge       = var.bridge
}

```

7.2 Fichier envs/B1-01/variables.tf

```

text
variable "pm_api_url"      { type = string }
variable "pm_api_token_id" { type = string }
variable "pm_api_token_secret" { type = string }

variable "vm_name"      { type = string default = "b1-01-vm1" }
variable "target_node" { type = string default = "pve1" }
variable "template_vmid" { type = number default = 900 } # à adapter
variable "vmid"         { type = number default = 110 }
variable "cores"        { type = number default = 2 }
variable "memory_mb"    { type = number default = 2048 }
variable "disk_size"    { type = string default = "20G" }

```

```
variable "storage"    { type = string default = "local-lvm" }  
variable "bridge"     { type = string default = "vmbr0" }
```

7.3 Fichier envs/B1-01/terraform.tfvars.example

```
text  
pm_api_url      = "https://PVE1_IP:8006/api2/json"  
pm_api_token_id = "terraform@pve!token_b1_01"  
pm_api_token_secret = "A_REEMPLACER"
```

Chaque groupe copiera ce fichier en terraform.tfvars et remplira ses secrets.

8. Utilisation par un groupe

Sur le Bastion, pour un groupe (ex. B1-01) :

```
bash  
cd terraform-lab/envs/B1-01  
  
# Créer terraform.tfvars à partir de l'exemple  
cp terraform.tfvars.example terraform.tfvars  
nano terraform.tfvars # remplir l'URL, le token, etc.  
  
# Initialiser le projet  
terraform init  
  
# Vérifier la configuration  
terraform validate  
  
# Voir le plan  
terraform plan
```

```
# Appliquer (créer la VM)
```

```
terraform apply
```

```
# Vérifier l'état
```

```
terraform show
```

À la fin, la VM b1-01-vm1 doit être visible dans Proxmox sur le nœud pve1.

9. Gestion des états et bonnes pratiques

- Le fichier terraform.tfstate est local au Bastion dans le dossier du groupe. Il ne doit pas être versionné dans Git.
- Ajouter un .gitignore à la racine du dépôt :

```
text
```

```
**/terraform.tfstate
```

```
**/terraform.tfstate.backup
```

```
**/.terraform/
```

```
**/terraform.tfvars
```

- Chaque groupe travaille dans son propre dossier envs/B*-** pour éviter les collisions de VMID et d'état.

10. Annexes – Liens de référence

Terraform

- Documentation générale Terraform
: <https://developer.hashicorp.com/terraform/docs>
- Tutoriels officiels : <https://developer.hashicorp.com/terraform/tutorials>
- Vue d'ensemble CLI
: <https://developer.hashicorp.com/terraform/cli/commands>

- Liste de commandes Terraform (cheatsheet)
: <https://gist.github.com/thesaravanakumar/884752979726713e2ee34e026539229e>
- Article FR "Maîtriser Terraform" : <https://blog.stephane-robert.info/docs/infra-as-code/provisionnement/terraform/>

Provider Proxmox

- GitHub Telmate / provider Proxmox
: <https://github.com/Telmate/terraform-provider-proxmox>
- Guide d'installation du provider : <https://github.com/Telmate/terraform-provider-proxmox/blob/master/docs/guides/installation.md>
- Ressource proxmox_vm_qemu : https://github.com/Telmate/terraform-provider-proxmox/blob/master/docs/resources/vm_qemu.md
- Exemple Proxmox 8 avec Terraform
: <https://www.trfore.com/posts/provisioning-proxmox-8-vms-with-terraform-and-telmate/>

Terraform + GitHub / CI

- Automatiser Terraform avec GitHub Actions (HashiCorp)
: <https://developer.hashicorp.com/terraform/tutorials/automation/github-actions>
- Article "Terraform in GitHub Actions" : <https://spacelift.io/blog/github-actions-terraform>
- Exemple de workflow Terraform sur GitHub
: <https://github.com/neillturner/terraform-github-actions>